

به نام خدا

* مجتبی دهقانی آرانی * گزارش تسک دو * کارآموزی (ترم تابستان) شرکت مهیمن *

مهندس وفایی

صورت تسک :

تسک ۲: ساخت Agent با چند Tool

هدف:

درک مفهوم agent و استفاده از tool ها

شرح تسک:

یک دستیار بساز که بتواند بسته به درخواست کاربر، از ابزار مناسب استفاده کند.

Tool های مورد نیاز:

- ماشین حساب (جمع، تفریق، ضرب، تقسیم)
- جستجو در متن (مثلاً همان FAQ)
- (اختیاری) نمایش زمان یا تاریخ

مراحل انجام:

- پیاده سازی هر tool به صورت تابع جداگانه
- تعریف ورودی و خروجی هر tool
- استفاده از مدل برای تشخیص نیاز کاربر
- انتخاب tool مناسب و اجرای آن
- ترکیب نتیجه و نمایش پاسخ نهایی

مثال ورودی:

- "۵ ضربدر ۱۲ چند میشه؟" → استفاده از ماشین حساب
- "شرایط ثبت نام چیه؟" → جستجو در متن
- "الان چه ساعتیه؟" → نمایش زمان

خروجی مورد انتظار:

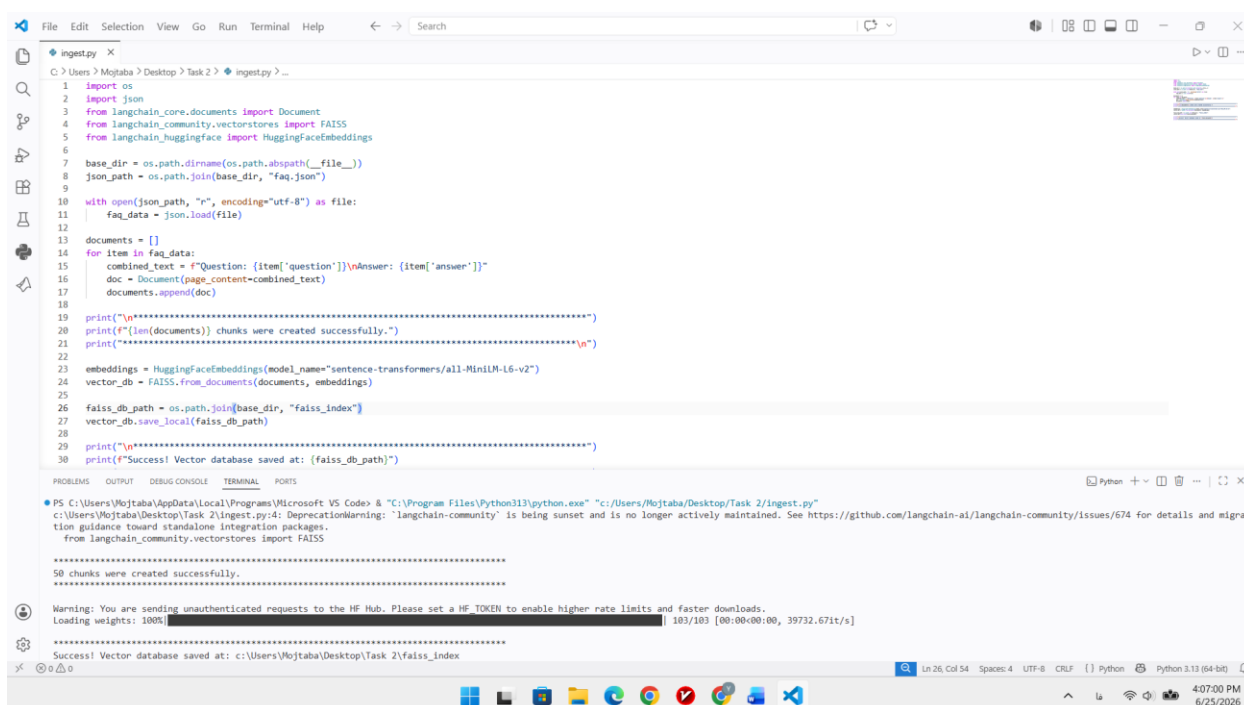
یک سیستم که بتواند تصمیم بگیرد چه ابزاری را استفاده کند و پاسخ مناسب بدهد

موارد پیشرفته تر:

- پرسیدن سوال در صورت مبهم بودن درخواست
- پاسخ مستقیم در صورت عدم نیاز به tool
- ثبت لاگ از تصمیم گیری ها

این تسک در ادامه تسک صفر و تسک یک زده شده است ، ما در تسک شماره یک ، یک چت بات ایجاد کردیم که فقط با توجه به فایل جیسون 50 عدد پرسش و پاسخ (فایل faq.json) ، به ما جواب میداد و خارج از این پرسش و پاسخ ها ، به ما میگفت ((نمیدانم)) ، اما حالا این چت بات ما می بایست مالتی تولز بشود ، به این معنی که چند تول داشته باشد و بر حسب نیاز و با توجه به خواسته و نیاز کاربر از tool های مختلف خودش استفاده کرده و به ما پاسخ بدهد .

چون کد این قسمت در ادامه کد تسک اول زده شده ما می بایست فایل ((.env)) و ((faq.json)) را در پوشه و در کنار کد ها قرار داده و کد ingest.py را نیز یک دور ران کنیم تا در پوشه ما ، فایل ((faiss_index)) ایجاد بشود .



```
1 import os
2 import json
3 from langchain_core.documents import Document
4 from langchain_community.vectorstores import FAISS
5 from langchain_huggingface import HuggingFaceEmbeddings
6
7 base_dir = os.path.dirname(os.path.abspath(__file__))
8 json_path = os.path.join(base_dir, "faq.json")
9
10 with open(json_path, "r", encoding="utf-8") as file:
11     faq_data = json.load(file)
12
13 documents = []
14 for item in faq_data:
15     combined_text = f"Question: {item['question']}\nAnswer: {item['answer']}"
16     doc = Document(page_content=combined_text)
17     documents.append(doc)
18
19 print("*****")
20 print(f"{len(documents)} chunks were created successfully.")
21 print("*****")
22
23 embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
24 vector_db = FAISS.from_documents(documents, embeddings)
25
26 faiss_db_path = os.path.join(base_dir, "faiss_index")
27 vector_db.save_local(faiss_db_path)
28
29 print("*****")
30 print(f"Success! Vector database saved at: {faiss_db_path}")
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100% | 103/103 [00:00:00:00, 39732.67it/s]

Success! Vector database saved at: c:\Users\Mojtaba\Desktop\Task 2\faiss_index

خب حالا به سراغ نوشتن کد Agent میرویم :

اول از همه بگم که من برای انجام این تسک ، مدل پایه خودم را از مدل "llama-3.3-70b-versatile"

به مدل "openai/gpt-oss-20b" تغییر دادم ، چون در طول انجام این تسک ، متوجه شدم که مدل

openai از لحاظ انعطاف پذیری در برابر پرامپت های سیستمی ، خیلی بهتر عمل میکند.

حالا تصاویری از کدی که زدم و خروجی هایی که گرفتم را میبینیم و سپس مختصری درباره منطق کدی که زدم و کاری که این کد دارد انجام میدهد ، توضیح می دهم :

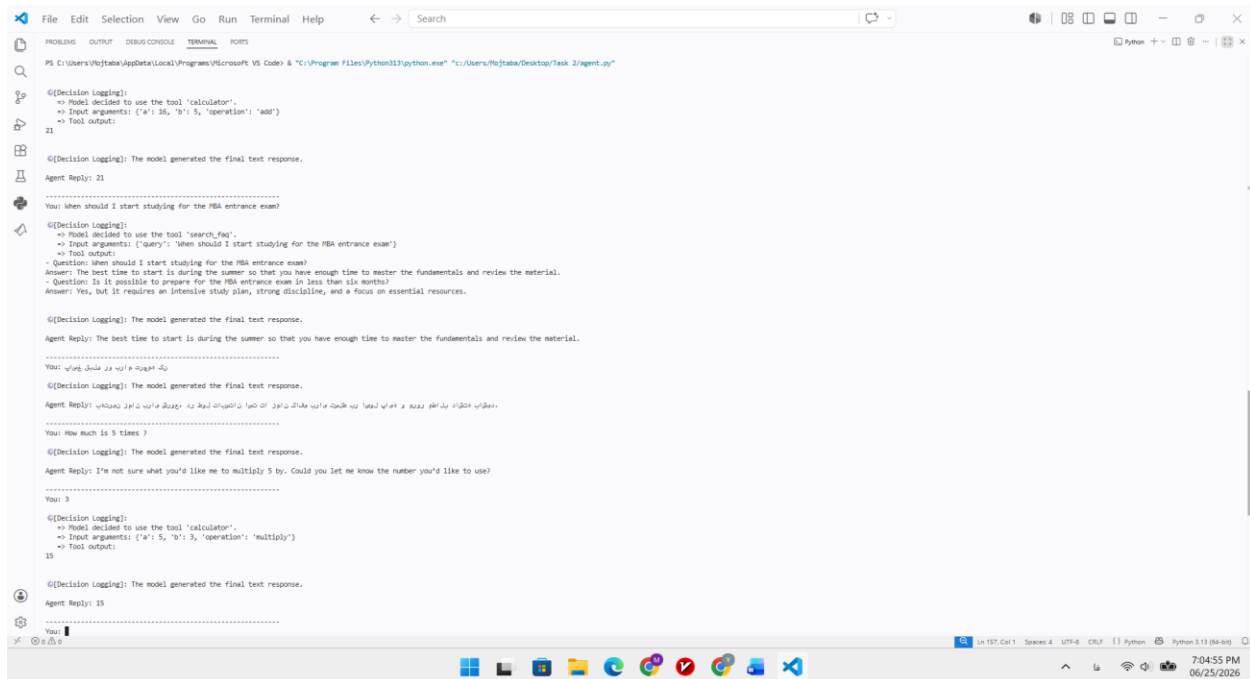
```
File Edit Selection View Go Run Terminal Help
agentay x [] Beqoon
C:\Users\Mohab> Desktop > Task 2 > agentay > @agent_chat
1 import os
2 import json
3 import datetime
4 from dotenv import load_dotenv
5 from openai import OpenAI
6 from langchain_community.vectorstores import FAISS
7 from langchain_huggingface import HuggingFaceEmbeddings
8
9 load_dotenv()
10 client = OpenAI(base_url="https://api.groq.com/openai/v1")
11
12 base_dir = os.path.dirname(os.path.abspath(__file__))
13 embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
14 faiss_db_path = os.path.join(base_dir, "faiss_index")
15
16 vector_db = FAISS.load_local(
17     faiss_db_path,
18     embeddings,
19     allow_dangerous_deserialization=True
20 )
21
22 def search_faq(query):
23     retrieved_docs = vector_db.similarity_search(query, k=2)
24     context = "\n".join([f"- {doc.page_content}" for doc in retrieved_docs])
25     return context if context else "No information found in the database."
26
27 def calculator(operation, a, b):
28     if operation == "add": return str(a + b)
29     elif operation == "subtract": return str(a - b)
30     elif operation == "multiply": return str(a * b)
31     elif operation == "divide":
32         return str(a / b) if b != 0 else "Error: Division by zero"
33     return "Invalid operation"
34
35 def get_time():
36     now = datetime.datetime.now()
37     return now.strftime("%Y-%M-%d %H:%M:%S")
38
39 tools = [
40     {
41         "type": "function",
42         "function": {
43             "name": "search_faq",
44             "description": "Used to search the database for rules, registration conditions, and FAQs. Extract and provide ONLY the exact answer to the user's specific question from the retrieved data. Do not include extra related information unless asked.",
45             "parameters": {
46                 "type": "object",
47                 "properties": {
48                     "query": {"type": "string", "description": "Search keyword or query"}
49                 },
50                 "required": ["query"],
51             },
52         },
53     },
54     {
55         "type": "function",
```

```
File Edit Selection View Go Run Terminal Help
agentay x [] Beqoon
C:\Users\Mohab> Desktop > Task 2 > agentay > ...
55     "type": "function",
56     "function": {
57         "name": "calculator",
58         "description": "Used to perform basic mathematical operations (add, subtract, multiply, divide) on two numbers. If the user only provides one number or the operation is unclear, do NOT guess. Ask the user for clarification.",
59         "parameters": {
60             "type": "object",
61             "properties": {
62                 "operation": {
63                     "type": "string",
64                     "enum": ["add", "subtract", "multiply", "divide"],
65                     "description": "Type of mathematical operation"
66                 },
67                 "a": {"type": "number", "description": "First number"},
68                 "b": {"type": "number", "description": "Second number"}
69             },
70             "required": ["operation", "a", "b"],
71         },
72     },
73 ],
74 {
75     "type": "function",
76     "function": {
77         "name": "get_time",
78         "description": "Used to get the current date and time.",
79         "parameters": {"type": "object", "properties": {}},
80     },
81 },
82 ]
83
84 SYSTEM_PROMPT = """You are a smart assistant (Agent) equipped with external tools.
85 Your tasks:
86 1. Analyze the user's request.
87 2. If a tool is needed, select the appropriate tool.
88 3. CRITICAL RULE (EXACT MATCH): When the 'search_faq' tool returns data, you MUST provide the exact 'Answer' text from the tool output verbatim. Do NOT translate, paraphrase, summarize, or change the formatting of the FAQ answer.
89 4. If the request is ambiguous, incomplete, or lacks necessary parameters, do NOT guess. Ask the user a clarifying question before taking any action.
90 5. If the question does not require a tool, answer directly.
91 6. CRITICAL RULE (LANGUAGE): Always reply in the exact same language as the user's CURRENT message. If the current question is in English, reply entirely in English. If it is in Persian, reply entirely in Persian. Do not mix languages."""
92
93 conversation_history = [{"role": "system", "content": SYSTEM_PROMPT}]
94
95 def agent_chat(user_input):
96     global conversation_history
97
98     conversation_history.append({"role": "user", "content": user_input})
99
100     try:
101         while True:
102             response = client.chat.completions.create(
103                 model="openai/gpt-oss-20b",
104                 messages=conversation_history,
105                 tools=tools,
106                 tool_choice="auto",
107                 temperature=0
108             )
109
```

```
File Edit Selection View Go Run Terminal Help
C:\Users\Mohaba\Desktop> Task 2 > agent.py --

95 def agent_chat(user_input):
123     print("=> input arguments: (function_args)")
124
125     if function_name == "search_faq":
126         function_response = search_faq(function_args.get("query"))
127     elif function_name == "calculator":
128         function_response = calculator(
129             function_args.get("operation"),
130             function_args.get("a"),
131             function_args.get("b")
132         )
133     elif function_name == "get_time":
134         function_response = get_time()
135     else:
136         function_response = "Tool not found."
137
138     print("=> Tool output:\n(function_response)\n")
139
140     conversation_history.append([
141         "tool_call_id": tool_call.id,
142         "role": "tool",
143         "name": function_name,
144         "content": function_response,
145     ])
146
147     else:
148         final_reply = response_message.content
149         if not tool_calls:
150             print("\n[Decision Logging]: The model generated the final text response.")
151
152         conversation_history.append(("role": "assistant", "content": final_reply))
153         return final_reply
154
155 except Exception as e:
156     return f"Error communicating with API: {e}"
157
158 if __name__ == "__main__":
159     print("\n" + "*" * 40)
160     print("Agent chatbot is running ... (Type 'exit' to quit)")
161     print("\n" + "*" * 40 + "\n")
162
163     while True:
164         user_text = input("You: ").strip()
165
166         if not user_text:
167             continue
168
169         if user_text.lower() in ['exit', 'quit']:
170             print("\nAgent: Goodbye! 🌹")
171             break
172
173         reply = agent_chat(user_text)
174         print("\nAgent: (reply)\n")
175         print("\n" + "*" * 40)
```

```
File Edit Selection View Go Run Terminal Help
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Mohaba\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python313\python.exe" "C:\Users\Mohaba\Desktop\Task 2\agent.py"
Agent chatbot is running ... (Type 'exit' to quit)
=====
You: HI
[Decision Logging]: The model generated the final text response.
Agent Reply: Hello! How can I help you today?
=====
You: ما اليوم
[Decision Logging]: The model generated the final text response.
Agent Reply: اليوم هو الاثنين 25 يونيو 2026
=====
You: ما اليوم والوقت
[Decision Logging]:
=> Model decided to use the tool 'get_time'.
=> Input arguments: {}
=> Tool output:
2026-06-25 18:45:55
[Decision Logging]: The model generated the final text response.
Agent Reply: اليوم 25 يونيو 2026
(الوقت 18:45:55)
=====
You: Tell me the date and time right now.
[Decision Logging]:
=> Model decided to use the tool 'get_time'.
=> Input arguments: {}
=> Tool output:
2026-06-25 18:46:06
[Decision Logging]: The model generated the final text response.
Agent Reply: Date: 2026-06-25
Time: 18:46:06 (local time)
=====
You: How are the subject coefficients weighted in the MBA entrance exam?
[Decision Logging]:
=> Model decided to use the tool 'search_faq'.
=> Input arguments: {'query': 'subject coefficients weighted MBA entrance exam'}
=> Tool output:
- Question: How are the subject coefficients weighted in the MBA entrance exam?
Answer: Typically, Mathematics, QWAT, and English all have equal weighting, each with a coefficient of one.
- Question: What subjects are included in the MBA entrance exam?
Answer: The exam consists of three main sections: Mathematics I & II, Academic Aptitude (QWAT), and Specialized English.
```



خب همونطور که قابل مشاهده هستش ، من سه Tool مختلف را ایجاد کرده ام که از عبارت هستند از :

1- جست و جوی هوشمند مبتنی بر RAG (search_faq) : این ابزار ، با استفاده از پایگاه داده برداری FAISS و مدل های Embeddings شرکت HuggingFace ، سوالات مرتبط کاربر را جست و جو کرده و مرتبط ترین بخش ها را برای پاسخ به سوالات متدوال استخراج میکند .

2- ماشین حساب (calculator) : این هم یک ابزار محاسباتی هستش برای انجام دقیق 4 عمل اصلی ریاضی با قابلیت مدیریت خطا (مثل خطای تقسیم بر صفر) .

3- گزارش کننده تایم (get_time) : این هم یک ابزار برای استخراج و نمایش تاریخ و ساعت لحظه ای سیستم هستش.

هسته اصلی برنامه تابع agent_chat هستش که از یک حلقه پردازشی (while True) استفاده میکند . در این حلقه، مدل زبانی ما درخواست کاربر را بررسی کرده ، ابزار مناسب را انتخاب میکند ، آرگومان های لازم را به فرمت JSON می سازد و پس از اجرای تابع در پایتون ، نتیجه را دریافت کرده و پاسخ نهایی متنی را تولید میکند . همچنین سیستم ما دارای قابلیت ثبت لاگ تصمیم گیری های ((Decision Logging)) برای دنبال کردن عملکرد مدل است . همچنین ، من از طریق ((System Prompt)) ، رفتار مدل را توسط یک دستور العمل سیستمی نسبتا دقیق کنترل میکنم که این پرامپت ، شامل قوانین کلیدی زیر هستش :

1- عدم تغییر یا خلاصه سازی متن استخراج شده در ابزار جست و جوی FAQ

2- جلوگیری از حدس زدن های اشتباه توسط مدل و الزام به پرسیدن سوال از کاربر در صورت وجود ابهام

3- تطبیق کامل زبان پاسخگویی با زبان ورودی کاربر به صورت خودکار